MAKAUT 4th Semester — Asansol Engineering College

Theory of Computation

CS 4th Sem | Exam: 18 June 2026 | Target: 9+ SGPA

Complete Study Notes + MAKAUT-Style Q&A;

UNIT 1: Finite Automata & Regular Languages

Q1. [5 Marks]

Define DFA. Construct a DFA to accept strings ending with '01' over Σ={0,1}.

Ans:

A **DFA (Deterministic Finite Automaton)** is a 5-tuple M = (Q, Σ, δ, q0, F) where:

• Q = finite set of states

• Σ = finite input alphabet

• δ: Q × Σ → Q = transition function (exactly one transition per state per input)

• q0 ∈ Q = initial/start state

• F ⊆ Q = set of final (accepting) states

DFA for strings ending in '01':

States: q0 (start, no useful prefix), q1 (last symbol was 0), q2 (last two symbols were 01 — final state).

Transitions:

δ(q0, 0) = q1 δ(q0, 1) = q0

δ(q1, 0) = q1 δ(q1, 1) = q2

δ(q2, 0) = q1 δ(q2, 1) = q0

Final states: F = {q2}

Test: '1001' → q0→q0→q1→q1→q2 ✓ (accepted). '010' → q0→q1→q2→q1 ✗ (rejected).

❖ *EXAM TIP: Always draw the transition diagram (bubble-and-arrow). Double circle = final state, arrow into q0 = start state. Verify with 2-3 test strings.*

Q2. [5 Marks]

What is an NFA? Convert NFA to DFA using Subset Construction.

Ans:

An **NFA (Nondeterministic Finite Automaton)** is a 5-tuple like DFA but δ: Q × Σ → 2^Q (can go to multiple states or none). Also allows ε-transitions.

NFA to DFA (Subset Construction Method):

The DFA states represent sets (subsets) of NFA states.

Algorithm:

1. Start: DFA start state = ε-closure(q0) of NFA.

2. For each DFA state (set of NFA states) and each input symbol a:

Compute: MOVE(T, a) = union of δ(q, a) for all q in T, then take ε-closure.

3. If result is a new state, add it to the DFA state table.

4. Mark DFA state as accepting if it contains any NFA accepting state.

5. Repeat until no new states created.

ε-closure: Set of states reachable from q using only ε-transitions (including q itself).

Key formula: ε-closure(MOVE(T, a))

> ✦ *EXAM TIP: For NFA→DFA questions: make a transition TABLE with columns for each input symbol. Each row = a DFA state (set). Examiners want to see the table, not just the diagram.*

Q3. [5 Marks]

Explain Regular Expressions. Prove that every Regular Expression has an equivalent FA.

Ans:

A **Regular Expression (RE)** describes a regular language using these operations:

• **Union:** r1 + r2 (or r1 | r2) — strings in r1 OR r2

• **Concatenation:** r1r2 — strings from r1 followed by r2

• **Kleene Star:** r* — zero or more repetitions of r

• **Plus:** r^+ = rr* (one or more)

Equivalences: RE ↔ NFA ↔ DFA ↔ Regular Grammar (all describe same class)

Common examples:

• (0+1)* = all binary strings

• (0+1)*01 = strings ending in 01

• 0*1* = zero or more 0s followed by zero or more 1s

• (01+10)* = even length strings over {0,1} with balanced structure

Thompson's Construction: Converts RE to NFA using ε-transitions for each operator.

Kleene's Theorem: A language is regular iff it is accepted by a DFA iff it can be described by a RE.

Q4. [5 Marks]

State and Apply the Pumping Lemma for Regular Languages.

Ans:

Pumping Lemma (Regular Languages): If L is a regular language, then there exists a constant p (pumping length) such that for every string $w \in L$ with $|w| \geq p$, w can be written as w = xyz satisfying:

1. $|xy| \leq p$

2. $|y| \geq 1$ (y is non-empty)

3. For all $i \geq 0$, $xy^iz \in L$

Use: To PROVE a language is NOT regular (proof by contradiction).

How to use:

1. Assume L is regular. So pumping length p exists.

2. Choose a string $w \in L$ with $|w| \geq p$.

3. Show that for ALL ways to split w=xyz (satisfying conditions 1,2), there exists some $i \geq 0$ such that $xy^iz \notin L$.

4. This contradicts the pumping lemma → L is not regular.

Classic Example: $L = \{0^n1^n \mid n \geq 1\}$

Assume regular. Let p be pumping length. Choose $w = 0^p1^p$. $|w| = 2p \geq p$.

Since $|xy| \leq p$, xy is entirely in the 0s part: $x = 0^a$, $y = 0^b$ ($b \geq 1$), $z = 0^{p-a-b}1^p$.

Pump i=2: $xy^2z = 0^{p+b}1^p$. Now we have more 0s than 1s → not in L. Contradiction!

Therefore L is NOT regular.

✦ *EXAM TIP: Pumping lemma proof is a TEMPLATE. Choose w carefully (usually 0^p 1^p or similar), split into xyz, and find i that breaks the pattern. Always state the contradiction explicitly.*

UNIT 2: Context-Free Languages & Grammars

Q5. [5 Marks]

Define CFG. Write a CFG for {a■b■ | n ≥ 1}. Explain Ambiguity.

Ans:

A **CFG (Context-Free Grammar)** is a 4-tuple G = (V, T, P, S) where:

• V = set of non-terminals (variables)

• T = set of terminals

• P = set of productions (rules) of form $A \to \alpha$ where $A \in V$, $\alpha \in (V \cup T)^*$

• $S \in V$ = start symbol

CFG for L = {a■b■ | n ≥ 1}:

$S \to aSb \mid ab$

Derivation: $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaabbb$ (for n=3)

Parse Tree: Tree representation of a derivation. Root = S, leaves = terminals.

Ambiguous Grammar: A grammar where some string has two or more different parse trees (or leftmost derivations).

Example: $E \to E+E \mid E*E \mid id$ is ambiguous.

String: id+id*id has two parse trees depending on which + or * you expand first.

Fixing ambiguity: $E \to E+T \mid T$; $T \to T*F \mid F$; $F \to id$. This enforces precedence.

Inherently Ambiguous Language: Some CFLs cannot be described by ANY unambiguous grammar.

Q6. [5 Marks]

Convert a CFG to Chomsky Normal Form (CNF).

Ans:

CNF (Chomsky Normal Form): Every production is of the form: $A \to BC$ (two non-terminals) or $A \to a$ (single terminal). No ε-productions (except possibly $S \to \varepsilon$).

Conversion Steps:

1. **START:** Add new start symbol S■ → S (if S appears on right side).

2. **TERM:** For each terminal a in bodies of length ≥ 2, create X■ → a and replace a with X■.

3. **BIN:** Break productions with ≥ 3 symbols on right into binary productions.

A → B C D becomes A → B■D■ and B■ → BC.

4. **DEL:** Remove ε-productions. Find all nullable variables; for each, add productions with and without that variable.

5. **UNIT:** Remove unit productions (A → B). Replace with all productions that B can derive.

Example: S → ASA | aB, A → B | S, B → b | ε

After CNF: S → AA■ | UB (X■ → a, etc.) — the full conversion is multi-step.

> ✦ *EXAM TIP: CNF conversion is always a 5-step process. Do in order: START → TERM → BIN → DEL → UNIT. State which step you're doing at each stage.*

Q7. [5 Marks]

Define PDA. Explain how PDA accepts {a■b■ | n ≥ 1}.

Ans:

A **PDA (Pushdown Automaton)** = NFA + stack (unlimited memory). Accepts all CFLs.

7-tuple: (Q, Σ, Γ, δ, q0, Z0, F) where Γ = stack alphabet, Z0 = initial stack symbol.

Transition notation: δ(q, a, X) = {(p, γ)} means: in state q, reading 'a' from input, with X on top of stack → move to state p and replace X with γ (γ can be ε for pop).

PDA for {a■b■}:

States: q0 (reading a's), q1 (reading b's), q2 (accept)

• δ(q0, a, Z0) = {(q0, AZ0)} [first 'a': push A, keep Z0]

• δ(q0, a, A) = {(q0, AA)} [subsequent 'a': push A]

• δ(q0, b, A) = {(q1, ε)} [first 'b': pop A, go to q1]

• δ(q1, b, A) = {(q1, ε)} [more 'b's: keep popping A]

• δ(q1, ε, Z0) = {(q2, Z0)} [stack empty (just Z0): accept]

Accepts by: final state. For 'aabb': Push A,A then Pop A,A → accept.

Two acceptance modes: By final state OR by empty stack (equivalent for CFLs).

UNIT 3: Turing Machines & Computability

Q8. [5 Marks]

Define Turing Machine. Design a TM for {a■b■ | n ≥ 1}.

Ans:

A **Turing Machine (TM)** is a 7-tuple M = (Q, Σ, Γ, δ, q0, B, F) where:

• Γ = tape alphabet (Σ ⊆ Γ, B ∈ Γ is blank)

• δ: Q × Γ → Q × Γ × {L, R} (state, read → new state, write, direction)

• Infinite tape, read/write head

TM for {a■b■}: Idea: Mark one 'a' with X, scan to first 'b', mark it Y, return to leftmost unmarked 'a', repeat.

States: q0 (scan right for a), q1 (scan right for b), q2 (scan left to start), q3 (check all marked), q_accept, q_reject

Key transitions:

• δ(q0, a) = (q1, X, R) [mark 'a' with X, move right]

• δ(q1, a) = (q1, a, R) [skip over a's]

• δ(q1, b) = (q2, Y, L) [mark 'b' with Y, move left]

• δ(q2, a) = (q2, a, L) [move left past a's]

• δ(q2, X) = (q0, X, R) [reached leftmost X, start again]

• δ(q0, Y) = (q3, Y, R) [no more a's, check remaining]

• δ(q3, Y) = (q3, Y, R) [skip Y's]

• δ(q3, B) = (q_accept, B, R) [all matched, accept]

Complexity: TM is the most powerful model of computation.

✦ *EXAM TIP: TM design questions: Explain the IDEA first (what the machine does conceptually), then write the transitions. Examiners give partial marks for correct logic even if transitions are incomplete.*

Q9. [5 Marks]

Explain the Halting Problem. Why is it undecidable?

Ans:

Halting Problem: Given a TM M and input w, does M halt (rather than loop forever) on w?

Claim: The halting problem is **undecidable** — no TM can solve it for all inputs.

Proof by Contradiction:

1. Assume there exists a TM H that decides the halting problem:

H(M, w) = accept if M halts on w; H(M, w) = reject if M loops on w.

2. Construct a new TM D that takes a TM description ■M■ as input:

D(■M■): Run H(M, ■M■). If H accepts (M halts on its own description) → D loops forever. If H rejects → D accepts.

3. Ask: What does D do on input ■D■?

If D halts on ■D■ → H says accept → D loops. Contradiction!

If D loops on ■D■ → H says reject → D accepts. Contradiction!

4. In both cases, contradiction. So H cannot exist. Halting problem is undecidable.

Implication: No program can automatically verify whether any other program will terminate.

Q10. [5 Marks]

Explain the Chomsky Hierarchy of Languages and Grammars.

Ans:

The **Chomsky Hierarchy** classifies formal languages into 4 types based on grammar restrictions:

Type	Grammar	Language	Automaton	Example
Type 0	Unrestricted: $\alpha \to \beta$ (any)	Recursively Enumerable	Turing Machine	All TM-decidable
Type 1	Context-Sensitive: $\alpha A \beta \to \alpha \gamma \beta$	Context-Sensitive	Linear Bounded Automaton	a■b■c■
Type 2	Context-Free: $A \to \gamma$	Context-Free (CFL)	PDA	a■b■

Type 3	Regular: A→aB or A→a	Regular	DFA/NFA	(01)*

Hierarchy: Type 3 ⊂ Type 2 ⊂ Type 1 ⊂ Type 0 (each is a proper subset of the next).